Email Notification Feature

This document describes the Sprint 9 email and account-security features: email verification with a soft gate, forgot and reset password, change password, self-service reactivation of closed accounts, a two-step email change, the Closed and Declined account statuses, and the account and review notice emails. It first explains how email is sent, then walks through each flow in order — route, service, table, email, link and page — and the nine `/auth` endpoints are then specified in the normal reference format. All paths are relative to `/api/v1`.

Every response uses the standard envelope: `{ "success": true, "message": "...", "data": ... }` on success and `{ "success": false, "message": "...", "error": { "code": "...", "details": ... } }` on failure. Authentication differs by endpoint: the emailed-link steps need no session, because the link is the proof, while the resend, change-password and change-email requests need `Authorization: Bearer <access token>`. Example responses show the message as "..." and concentrate on the data; all example data is fictional.

1. Overview

Sprint 9 added email to PetPals, and with it a set of account-security features built on emailed single-use links:

- **Email verification with a soft gate** — every account has an `emailVerified` flag. Registering sends the first link, and anyone can log in unverified. Only applying to adopt and donating need a verified email (409 `email_unverified`). (Sections 4.1 and 8.10)
- **Forgot and reset password** — **Forgot password?** on either login page emails a link that lasts one hour. Setting the new password signs out every session. (Section 4.2)
- **Change password** — while signed in, with the current password. This session stays signed in, any other session is signed out, and a notice is emailed. (Section 4.3)
- **Self-service reactivation** — an owner who closed their account can reopen it from an emailed link. Adopter and Donor accounts go straight back to Active; Staff, Veterinarian, Volunteer and Admin accounts go back to Pending and are re-approved. (Section 4.4)
- **Two-step email change** — a confirmation link goes to the new address, a heads-up goes to the old one, and nothing changes until the link is opened. (Section 4.5)
- **Closed and Declined statuses** — Closed (the owner closed the account) is kept apart from Deactivated (done by an admin or staff member) and from Declined (turned down at approval). That is what lets a reactivation link work only for accounts their owner closed. (Section 5)
- **Account and review notices** — emails about the account itself (closed, deleted, approved, declined, deactivated, banned, restored, password or email changed) and about review outcomes (government ID verified or rejected, adoption application approved or declined). (Section 6)
- **Email is optional** — with `EMAIL_API_KEY` or `EMAIL_FROM` unset nothing is sent. Every email, link included, is printed to the server console, so every flow works locally. (Sections 2.4 and 9)

> ***Scope decision — what is not emailed:*** *application received, visits, tasks and events, appointments, donation receipts and reminders. The 18 emails in Section 6 cover account security and review outcomes only.*

The table below lists every endpoint in this document, and the walkthrough or reference entry that covers it.

Endpoint	Auth	Purpose
`POST /auth/email-verification`	Bearer	Email a new verification link (4.1, 8.1)
`PATCH /auth/email-verification`	None (link)	Verify the address with the link's token (4.1, 8.2)
`POST /auth/password-reset`	None	Forgot password: email a reset link (4.2, 8.3)
`PATCH /auth/password-reset`	None (link)	Set a new password with the link's token (4.2, 8.4)
`PATCH /auth/password`	Bearer	Change password while signed in (4.3, 8.5)
`POST /auth/reactivation`	None	Email a link to reopen a Closed account (4.4, 8.6)

Endpoint	Auth	Purpose
`PATCH /auth/reactivation`	None (link)	Reopen the account with the link's token (4.4, 8.7)
`POST /auth/email-change`	Bearer	Ask to change the login email (4.5, 8.8)
`PATCH /auth/email-change`	None (link)	Confirm the new email with the link's token (4.5, 8.9)
`POST /adoption-applications`, `POST /donations/checkout`	Bearer	The two actions behind the verification gate (4.1, 8.10)

> ***One mechanism, four links:*** *verification, reset, reactivation and email-change links are all single-use tokens managed by one service (Section 3). Each endpoint pair differs only in what it changes once the token is accepted.*

2. Architecture of Sending (services/email/)

Everything that sends email goes through one small layer, `services/email/`. Callers build a message, hand it to `queueEmail` after their database write has committed, and carry on; the layer decides whether it goes to Brevo or to the console.

2.1 Structure

`services/email/index.js` is the only import for email. It exports `isEmailConfigured`, `sendEmail`, `queueEmail` and `templates`. `brevoApi.js` is the only file that talks to the provider, and `templates.js` holds one function per email. This is the same pattern as the project's `services/ai/`, `services/geocoding/` and `services/storage/`: moving to another HTTPS provider (Resend, Postmark and so on) means rewriting `brevoApi.js` alone.

File	Responsibility
`services/email/index.js`	The only import. `sendEmail` sends one message, `queueEmail` sends it fire-and-forget, and `templates` re-exports every email.
`services/email/brevoApi.js`	The only file that talks to the provider: `isEmailConfigured()` and `sendViaProvider()`.
`services/email/templates.js`	One function per email, all built by `renderEmail()`, plus `clientLink()` for links into the frontend.
`config/email.js`	Link lifetimes, the resend cooldown, the provider timeout and the default sender name (Section 9).
`services/auth/accountNotices.js`	Emails about the account itself: closed or deleted, and status changes someone else made (Section 6).
`services/staff/reviewNotices.js`	Emails about a review someone else made: government ID, adoption application (Section 6).
`services/auth/userContact`	Looks up the email, name and status of any role's account, for the notices above.

2.2 Provider: Brevo's HTTPS API

`sendViaProvider` POSTs one JSON request to `https://api.brevo.com/v3/smtp/email` with the header `api-key: <EMAIL_API_KEY>`. The body carries `sender` ({ email, name }), `to` ([{ email }]), `subject`, `htmlContent` and `textContent`. It uses Node's global `fetch`, with no SDK.

> ***Why HTTPS and not SMTP:*** *hosting platforms often block outbound SMTP, especially on free plans, and an HTTPS API works everywhere. It also means no email SDK dependency.*

2.3 Configuration is read on every call

`brevoApi.js` reads `process.env` on every call, not at require time, so tests can switch email on and off by setting variables. `EMAIL_API_KEY` and `EMAIL_FROM` are trimmed. `EMAIL_FROM` must be a sender verified in the Brevo account, or Brevo rejects the send. The sender name is `EMAIL_FROM_NAME`, or `DEFAULT_FROM_NAME` ("PetPals") when that is unset. `isEmailConfigured()` is true only when both the key and the sender address are non-empty.

2.4 Console fallback

`sendEmail(message)` takes `{ to, subject, text, html }` (usually a template's output plus `to`). With email not configured it prints the email to the server console, link included, and returns `{ delivered: false }`; otherwise it calls `sendViaProvider` and returns `{ delivered: true }`. The console output starts with `[email] not configured — would send to <address>`, followed by the subject and the plain-text body. When `NODE_ENV` is `test` nothing is printed, so tests stay quiet and assert on a mocked module instead.

2.5 queueEmail: fire-and-forget after the commit

`queueEmail(message)` runs `sendEmail` in the background and is never awaited. Callers use it after their database write has committed. If the send fails, the error is logged as `[email] failed to send "<subject>" to <address>: <reason>` and never thrown: the request it belongs to has already succeeded, and a slow provider must not hold up the response.

2.6 Provider timeout and failures

`sendViaProvider` gives up after `SEND_TIMEOUT_MS` (10000 ms, from `config/email.js`) using `AbortSignal.timeout`, so a hung connection can't linger. It resolves once Brevo has accepted the email and otherwise throws:

Failure	Error message
No reply within the timeout	Email provider timed out after <ms> ms
Network failure	Email provider unreachable: <reason>
Non-2xx reply	Email provider returned <status>, plus the provider's own message when it sent one. The request that was sent is never included.

These errors only ever reach the server log, because the send is queued after the response work is done.

2.7 Templates and escaping

Every email is built by `renderEmail({ subject, heading, paragraphs, action, footnote })`, which returns `{ subject, text, html }`. All emails therefore share one plain layout, and each comes as both HTML and plain text, because some inboxes show only the text part. `action` is an optional `{ label, url }`: a button in the HTML and the bare URL in the text. Every value is passed through `escapeHtml` (`&`, `<`, `>`, `"` and `'`), so names and staff remarks can go in as they are. The greeting is `Hi <name>,`, or `Hi,` when the account has no name yet, and every email ends "— The PetPals team". `clientLink(path, params)` builds a link into the frontend from `CLIENT_URL` (default `http://localhost:3000`), for example `clientLink("/verify-email", { token })`.

Diagram — sending one email

```
service writes to the database
(the $transaction commits)
      |
      v
message = templates.verifyEmail({ name, url })
      -> { subject, text, html }
      |
      v
queueEmail({ to, ...message })    never awaited
      |
      v
sendEmail(message)
      |
      +-- EMAIL_API_KEY and EMAIL_FROM both set
      |        |
      |        v
      |   brevoApi.sendViaProvider()
      |   POST api.brevo.com/v3/smtp/email
      |     +-- accepted: { delivered: true }
      |     +-- timeout, network error or non-2xx:
      |         logged, never thrown
      |
      +-- either one unset
               |
               v
           printed to the server console
           (link included): { delivered: false }
```

> ***Order of the calls:*** *the caller builds the message with a template first and then passes it to* `queueEmail`*. The queue never builds content itself.*

3. Emailed Links (AccountToken)

Four emails carry a link that does something when opened: verify an email address, reset a password, reactivate a Closed account and confirm an email change. All four use one service, `services/auth/accountTokens.service.js`, and one table, `AccountToken`. Full column definitions are in 04 - Database Design (3.32).

3.1 Token lifecycle

Step	What happens
Issue	`issueToken(userID, purpose)` creates a random token (32 random bytes, base64url). The raw token goes into the emailed link and nowhere else: it is never in a response and never in the database.
Hash	Only the SHA-256 hex of the token is stored (`tokenHash`, VARCHAR(64), unique). A copy of the table can't be turned back into working links.
Newest only	Issuing deletes the user's earlier unused tokens of the same purpose, plus any of their expired ones, so only the newest link works.
Cooldown	A new link of the same purpose can't be requested within 60 seconds. `issueToken` then throws TOO_MANY_REQUESTS (429) with `error.details.retryAfterSeconds`, before anything is deleted, so the earlier link keeps working.

Step	What happens
Consume	`consumeToken(rawToken, purpose)` trims and hashes the token, finds its row and marks it used with a conditional update, so it is single-use even when two requests race. It returns `{ userID, newEmail }`.
Same transaction	Both functions accept the caller's Prisma `$transaction` client. Each endpoint consumes its token inside the same transaction as the change the link authorises, so a failed change leaves the link usable.
Cleanup	`AccountToken` rows cascade with their user (`ON DELETE CASCADE` from Users), so deleting an account clears its tokens.

Diagram — one link, issue to consume

```
REQUEST A LINK  (POST ...)
      |
      v
issueToken(userID, purpose)
 - last link under 60 s ago?  -> 429
 - delete earlier unused tokens of this
   purpose, and expired ones
 - new random token; store SHA-256 only
      |
      v
the email carries the raw token in its link
      :
      :   the person opens the link  (PATCH ...)
      :
      v
consumeToken(rawToken, purpose)
 - missing, unknown, other purpose -> token_invalid
 - already used                    -> token_used
 - past its expiry                 -> token_expired
 - otherwise: mark used (conditional update)
      |
      v
the change, in the SAME $transaction
```

3.2 Failure reasons

`consumeToken` checks in this order and throws a 400 BAD_REQUEST whose `error.details.reason` lets the client tell "request a new link" from "already done":

error.details.reason	When	Message
`token_invalid`	The token is missing or not a string, is unknown, or was issued for a different purpose	This link is invalid
`token_used`	The link was already redeemed, or a racing request redeemed it first	This link has already been used
`token_expired`	The link is past its lifetime	This link has expired — please request a new one

The 429 from the cooldown reads: "An email was just sent — please wait N seconds before asking for another".

3.3 Lifetimes per purpose

Purpose	Lifetime	Notes
`EmailVerification`	24 hours	Registering sends the first link; the resend replaces it
`PasswordReset`	1 hour	The shortest life: a reset link sets a new password without the old one
`Reactivation`	24 hours	Only issued for a Closed account
`EmailChange`	24 hours	The only purpose that stores `newEmail` on its row

The lifetimes (`TOKEN_TTL_MS`) and the cooldown (`RESEND_COOLDOWN_MS`, 60 seconds) are defined once in `config/email.js`. The cooldown stops someone hammering "Resend" into a stranger's inbox.

3.4 Why opening a link also verifies the address

Opening a link proves the person can read mail at that address. So a password-reset, reactivation or email-change link also sets `emailVerified` to true, in addition to the verification link itself. For an email change the new address is the one marked verified.

4. Flow Walkthroughs

Each flow below is a short numbered sequence: what the person does, the endpoint, what is written, the email that is sent, the link, the page and the result. Every email is queued after the write commits (Section 2.5), and every token step follows Section 3.

4.1 Email verification and the soft gate

Flow — verify an email address

1. **Sign up:** `POST /auth/register` creates the account and sends the first verification link. Accounts that existed before Sprint 9 were backfilled as verified, so the gate below doesn't suddenly block them.
2. **Link issued:** `issueToken(userID, "EmailVerification")` stores the token's hash and the email carries a link to `/verify-email?token=...`. The link works once and expires in 24 hours.
3. **Resend:** the dashboard banner, the Profile and the apply/donate notice all offer a Verify or Resend button, which calls `POST /auth/email-verification` (Bearer; a Pending Staff member, Veterinarian or Volunteer may call it too). A new link replaces the previous unused one. Already verified returns 409 `already_verified`; asking again within 60 seconds returns 429.
4. **Open the link:** the `/verify-email` page redeems `?token=` once, as soon as it opens, with `PATCH /auth/email-verification { token }`. No session is needed, so the link can be opened on another device.
5. **Result:** `USERS.emailVerified` becomes true and the response is `{ emailVerified: true }`. In a signed-in tab the session is updated and the queries are refreshed, so the banner disappears.

The soft gate

Everyone can log in with an unverified email and use their dashboard, where a banner offers to send the link. Only two actions need a verified email: applying to adopt (`POST /adoption-applications`) and donating (`POST /donations/checkout`). Both answer 409 `CONFLICT` with `error.details.reason = "email_unverified"`, and the check runs before any pet or shelter lookup and before any Stripe call.

> ***Why 409 and not 403:*** *the frontend's axios interceptor sends every 403 to `/forbidden`, so a 403 would send the person away from the page instead of letting it ask them to verify. On the client, the apply and donate forms replace their submit button with a notice, and a 409 `email_unverified` also flips the session's `emailVerified` (Section 7).*

4.2 Forgot password and reset

Flow — forgot password → reset

1. **Ask: Forgot password?** on either login page opens `/forgot-password` (`?portal=staff` keeps the worker look). The page calls `POST /auth/password-reset { email }`.
2. **Same answer for everyone:** the response is always 200 with the same message, whether or not the email has an account, so it can't be used to find out who is registered. A link is only sent to an existing account that isn't Banned or Declined, at most once a minute (a quicker repeat sends nothing).
3. **Email:** `issueToken(userID, "PasswordReset")` replaces any earlier unused reset link, and the "Reset your PetPals password" email carries a link to `/reset-password?token=...` that works once and expires in 1 hour.
4. **Choose a password:** the `/reset-password` page is the one link page that doesn't redeem on load, because it needs the new password first. The password must meet the five rules (Section 8.4).
5. **Redeem:** `PATCH /auth/password-reset { token, password }` consumes the token in the same transaction as the password change. A weak password returns 422 with `error.details.unmet`, and a failed change leaves the link usable.
6. **Result:** every session is signed out (the stored refresh token and the cookie are cleared), the email is marked verified, a "password changed" notice is emailed, and `{ role }` is returned so the page can link to the right login.

4.3 Change password while signed in

Flow — change password

1. **Ask:** Profile → **Account & security** → change password. The page calls `PATCH /auth/password { currentPassword, newPassword }` with the access token (a Pending Staff member, Veterinarian or Volunteer can call it while onboarding).
2. **Checks:** a wrong current password returns 401 `wrong_password`; a new password that is weak, or the same as the current one, returns 422.
3. **Result:** this session stays signed in (a new refresh token cookie is set) and any other session is signed out.
4. **Email:** a "Your PetPals password was changed" notice goes to the account's address, with a button to `/forgot-password` in case it wasn't them.

4.4 Close an account, then reactivate it

Flow — close → login says closed → reactivation link

1. **Close:** the owner closes the account with `DELETE /<role>/me { mode: "deactivate" }` (every role). `accountStatus` becomes **Closed**, the owner is signed out everywhere and the client ends the session.
2. **Email:** an "Your PetPals account is closed" notice is queued after the commit. It tells the owner they can log in and ask for a reactivation link. (With `mode: "delete"` the account is removed instead, and an "account was deleted" notice is sent.)
3. **Login says closed:** logging in with the correct password returns 401 with `error.details.reason = "account_closed"`. The login page shows `ReactivationOffer`, with an **Email me a reactivation link** button.
4. **Ask for the link:** `POST /auth/reactivation { email }` always returns the same 200. A link is only sent when the email belongs to a Closed account (never to one an admin or staff member deactivated, one declined at sign-up, or a banned one), at most once a minute. It expires in 24 hours.
5. **Open the link:** the `/reactivate` page redeems `?token=` once on load with `PATCH /auth/reactivation { token }`. The token is consumed in the same transaction as the status change.
6. **Result:** the account is reopened, the email is marked verified and the response is `{ role, accountStatus }`. If the account is no longer Closed the answer is 409 `not_closed` and the link stays unused.

Reactivation outcome by role

Role	Status after	What happens next
Adopter	Active	Goes straight back to Active
Donor	Active	Goes straight back to Active
Staff	Pending	Re-approved the usual way
Veterinarian	Pending	Re-approved the usual way
Volunteer	Pending	Re-approved the usual way
Admin	Pending	Re-approved the usual way; a Pending Admin can't log in

4.5 Change email

Flow — change email

1. **Ask:** Profile → **Account & security** → change email. The page calls `POST /auth/email-change { newEmail, currentPassword }` with the access token (plain `authenticate`, so a Pending account gets 401).
2. **Checks:** a wrong password is 401 `wrong_password`; an address another account uses is 409 `email_taken`; a missing, invalid, longer-than-45-character or unchanged address is 422; asking again within 60 seconds is 429.
3. **Token:** `issueToken(userID, "EmailChange", { newEmail })` stores the new address on the token row. A newer request replaces it, and the link expires in 24 hours. Nothing about the account changes yet.
4. **Two emails:** the "Confirm your new PetPals email address" email goes to the **new** address, which proves the person can receive mail there. A heads-up, "A request to change your PetPals email address", goes to the **old** address, with a button to reset the password, so a hijacked session can't quietly move the account.
5. **Confirm:** the `/confirm-email-change` page redeems `?token=` once on load with `PATCH /auth/email-change { token }`. It re-checks that the address is still free (409 `email_taken` if someone took it meanwhile) and consumes the token in the same transaction as the update.
6. **Result:** the new address becomes the login email (verified) and the response is `{ userEmail }`. The old address gets a "Your PetPals email address was changed" notice. In a signed-in tab the session is updated and the queries are refreshed.

5. Account Statuses

Each role table has an `accountStatus` column. Sprint 9 added Closed to every role's status enum and Declined to the four roles that need approval (Staff, Veterinarian, Volunteer and Admin). The values have the same meaning in every role's enum. Column definitions and the migrations are in 04 - Database Design (3.33 and 3.35).

5.1 The six statuses

Status	Set by	Who can change it	What login says
Pending	Sign-up of Staff, Veterinarian, Volunteer and Admin accounts, which wait for approval	An approver: Pending → Active or Declined	Staff, Veterinarian and Volunteer can log in to onboard before approval, but reach only their own onboarding routes. A Pending Admin gets 401.

Status	Set by	Who can change it	What login says
Active	Approval; for Donors, sign-up; reactivation (Adopter, Donor); a restore by someone else	Admin or staff member, for the changes in 5.2	Login succeeds
Deactivated	An admin or staff member	Only an admin or staff member (Deactivated → Active, admin endpoints)	Refused with 401; keeps its own message
Banned	Someone other than the owner; used for Adopter, Donor and Volunteer accounts	Someone other than the owner (a restored notice is emailed)	Refused with 401; keeps its own message
Closed	The owner, through `DELETE /<role>/me` with mode `deactivate`	Only the owner, through the emailed reactivation link. Nobody else can change it (409).	Refused with 401 after a correct password; `error.details.reason = "account_closed"`, so the login page offers a reactivation link
Declined	Turned down at approval (Staff, Veterinarian, Volunteer, Admin)	Nobody. Declined is final: the account can't be approved later.	Refused with 401 after a correct password; `error.details.reason = "account_declined"`

> ***Why Closed and Declined were split out of Deactivated:*** *a self-service reactivation link must only work for accounts their owner closed. Closed is the one status its owner can undo, Deactivated can only be lifted by an admin or staff member, and Declined is final. Before the split these cases shared Deactivated.*

5.2 Allowed transitions

Changes made by someone other than the owner follow a fixed set of transitions; anything else returns 409. Closing an account and reactivating it are the owner's own actions and are listed here for completeness.

Transition	Done by	Applies to	Email sent
Pending → Active	An approver (approval)	Staff, Veterinarian, Volunteer, Admin	Account approved
Pending → Declined	An approver	Staff, Veterinarian, Volunteer, Admin	Sign-up declined
Active → Deactivated	Admin or staff member	Every role	Account deactivated
Deactivated → Active	Admin endpoints	Every role	Account active again
Active, Banned and Deactivated among themselves	Someone other than the owner	Adopter	The notice for the new status: banned, deactivated or restored
Closed (owner action)	The owner	Every role	Account closed
Closed → Active	The owner, by emailed link	Adopter, Donor	None; the link email itself
Closed → Pending	The owner, by emailed link	Staff, Veterinarian, Volunteer, Admin	None; the link email itself. Re-approved the usual way.

In `utils/staffDates.js`, Closed stamps `staffDOS` the same way Deactivated does; Declined stamps nothing. Admin rows whose own deactivation was on record were relabelled Closed by the Sprint 9 migration; every other Deactivated row stays Deactivated, because for those the owner and an admin or staff member can't be told apart.

6. Notice Emails

Eighteen emails exist, all built in `templates.js`. They fall into three groups, by the file that sends them. Every one is queued after the write that caused it has committed, and none of them can fail the request (Section 2.5).

6.1 Link emails (auth services)

These carry a link (Section 3) or tell the owner about a change they made themselves. They are sent from the `services/auth/` services for email verification, passwords, reactivation and email change, and from register.

Email	Subject	Sent to	Trigger	Sent by
`verifyEmail`	Verify your PetPals email address	The account's address	Registering; `POST /auth/email-verification`	Register; emailVerification service
`passwordReset`	Reset your PetPals password	The account's address	`POST /auth/password-reset`, for an existing account that isn't Banned or Declined	passwords service
`passwordChanged`	Your PetPals password was changed	The account's address	`PATCH /auth/password-reset`; `PATCH /auth/password`	passwords service
`reactivateAccount`	Reactivate your PetPals account	The Closed account's address	`POST /auth/reactivation`, for a Closed account	reactivation service
`confirmEmailChange`	Confirm your new PetPals email address	The new address	`POST /auth/email-change`	emailChange service
`emailChangeRequested`	A request to change your PetPals email address	The old address	`POST /auth/email-change`	emailChange service
`emailChanged`	Your PetPals email address was changed	The old address	`PATCH /auth/email-change`	emailChange service

6.2 Account notices (accountNotices.js)

`services/auth/accountNotices.js` sends emails about the account itself: the owner closing or deleting it, and status changes someone else made. `loginUrlFor(role)` picks the login page in the email: workers (Admin, Staff, Veterinarian) get `/staff-portal/login`, everyone else `/login`.

Email	Subject	Sent to	Trigger	Sent by
`accountClosed`	Your PetPals account is closed	The owner	`DELETE /<role>/me`, mode `deactivate` (→ Closed)	accountNotices.js, `accountClosed`
`accountDeleted`	Your PetPals account was deleted	The owner	`DELETE /<role>/me`, mode `delete`	accountNotices.js, `accountClosed`
`accountApproved`	Your PetPals account is approved	The account holder	Status change Pending → Active	accountNotices.js, `statusChanged`
`accountDeclined`	Your PetPals sign-up	The account holder	Status change Pending → Declined	accountNotices.js, `statusChanged`

Email	Subject	Sent to	Trigger	Sent by
`accountDeactivated`	Your PetPals account was deactivated	The account holder	Status change to Deactivated	accountNotices.js, `statusChanged`
`accountBanned`	Your PetPals account was suspended	The account holder	Status change to Banned	accountNotices.js, `statusChanged`
`accountRestored`	Your PetPals account is active again	The account holder	Status change to Active from Deactivated or Banned	accountNotices.js, `statusChanged`

`statusChanged(userID, from, to)` is called after an admin or staff status change on the six status endpoints. It maps the change to a template with `statusTemplate`, looks the contact up itself (the account still exists) and sends nothing for a change that has no notice. For the owner's own close or delete, `lookupContact` reads the contact **before** a delete removes the rows, and returns null if the lookup fails.

6.3 Review outcomes (reviewNotices.js)

`services/staff/reviewNotices.js` sends emails about a review someone else made. It has the same contract as the account notices: called after the write commits, and a failed lookup or send is logged, never thrown.

Email	Subject	Sent to	Trigger	Sent by
`governmentIdVerified`	Your government ID is verified	The ID's owner	`PATCH /government-ids/:id/status` to Verified	reviewNotices.js, `governmentIdReviewed`
`governmentIdRejected`	Your government ID needs another look	The ID's owner	`PATCH /government-ids/:id/status` to Rejected	reviewNotices.js, `governmentIdReviewed`
`applicationApproved`	Your application for <pet> was approved	The applicant	Staff or admin accepts the application	reviewNotices.js, `applicationsDecided`
`applicationDeclined`	An update on your application for <pet>	The applicant	Staff or admin rejects the application, or it is declined automatically (below)	reviewNotices.js, `applicationsDecided`

- **Pending accounts** — when the ID's owner is still Pending (a worker or volunteer onboarding), the verified email adds that the account is still waiting for approval, and the rejected email points to the approval page for the new upload. Everyone else is pointed to their Profile.
- **Wording** — adopter-facing wording matches the dashboard: Accepted reads "approved" and Rejected reads "declined". When the shelter left a remark, the email adds it as "A note from the shelter".
- **Automatic decline** — when one application for a pet is accepted, the pet's other Pending applications are declined automatically with an automatic staff remark. Each of those applicants gets the declined email. The caller finds them with a `findMany` inside the same transaction, so `applicationsDecided` receives the decided one plus the automatic declines as one list and needs no lookup of its own.

6.4 What is not emailed

Application received, visits, tasks and events, appointments, donation receipts and reminders do not send email. That is a scope decision, not a gap: every email in this section is either about the account's security or the outcome of a review.

7. Client Side

The frontend needs `emailVerified` in the session, a way to nudge unverified people, five landing pages for the links, and an Account & security section on every Profile.

7.1 Verification prompts

- **Session** — `AuthUser.emailVerified` comes back from login and refresh-token, so session restore on page load populates it.
- **Dashboard banner** — `EmailVerificationBanner` sits at the top of `<main>` in the shared `DashboardLayout`, so every role's dashboard shows it.
- **Verify button on every Profile** — `EmailVerificationStatus` shows the status with a Verify button. It takes `onVerify` and `pending` props and has no API import of its own.
- **Resending** — every Verify and Resend button goes through `useResendVerification`: it shows the toasts, treats a 409 `already_verified` as "the session catches up", and shows the wait on a 429.
- **Apply and donate notice** — `EmailUnverifiedNotice` (in `protected/shared/`) replaces the submit button in `AdoptApply` and `DonateForm`. A 409 `email_unverified` from the server also flips the session's `emailVerified`.

7.2 Link landing pages

The five pages live in `pages/public/auth/`. They redeem the `?token=` once when they mount (a ref guard stops React StrictMode redeeming it twice), except the reset page, which needs the new password first.

Page	Route	Calls	Notes
`VerifyEmail`	`/verify-email`	`PATCH /auth/email-verification` on load	Works without a session
`ForgotPassword`	`/forgot-password`	`POST /auth/password-reset`	`?portal=staff` keeps the worker look
`ResetPassword`	`/reset-password`	`PATCH /auth/password-reset` after the new password	Doesn't redeem on load
`Reactivate`	`/reactivate`	`PATCH /auth/reactivation` on load	Shown after the email link
`ConfirmEmailChange`	`/confirm-email-change`	`PATCH /auth/email-change` on load	Opened from the new address

`Login` and `WorkerLogin` add a **Forgot password?** link and, when the server answers `account_closed`, `ReactivationOffer`. The shared `AuthCard` gives all of these pages the same background, card and field classes. After a verification or an email change in a signed-in tab, the session is updated with `updateUser(...)` and `queryClient.invalidateQueries()` runs.

7.3 Account & security section

`AccountSecuritySection` (in `protected/shared/`) holds change password and change email. It is on all six Profiles, above the Danger zone.

7.4 Why link pages skip the onboarding redirect

`OnboardingGate` redirects a signed-in user to their onboarding wizard, or to the awaiting-approval page for a Pending account. All five link pages are listed in the gate's `LINK_PAGES`, so a user who is mid-onboarding or Pending isn't redirected away before the token in the link has been used. That matters because verification is allowed for Pending Staff, Veterinarians and Volunteers while they onboard, and a link can be opened at any point. Any new link page must be added to `LINK_PAGES`.

8. API Reference

The conventions below apply to every endpoint in this section. Paths are relative to `/api/v1`. Where an endpoint needs a session, the request carries `Authorization: Bearer <access token>`. Every `error.details.reason` named here is an exact string the client can switch on.

Error Code	HTTP Status	Meaning
BAD_REQUEST	400	An emailed link is invalid, expired or already used
UNAUTHORIZED	401	Not signed in, or the current password is wrong
CONFLICT	409	Already verified, address taken, account no longer Closed, or email not verified
VALIDATION_ERROR	422	A required field is missing or invalid, or the password is too weak
TOO_MANY_REQUESTS	429	Another emailed link was asked for within 60 seconds

A password is accepted only if it has 8 or more characters with an uppercase letter, a lowercase letter, a number and a special character. The same five rules apply on register, reset and change. A weak password returns 422 with `error.details.unmet` listing the missing rules.

8.1 POST /api/v1/auth/email-verification

Emails the signed-in user a new verification link. Registration already sends one; this is the resend. Each new link replaces the previous unused one and expires in 24 hours.

Request

Field	Value
Method	POST
Auth required	Yes — Bearer access token, through `authenticate.allowPending`, so a Pending Staff member, Veterinarian or Volunteer is admitted
Body fields	None

Example request

```
POST /api/v1/auth/email-verification
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in (missing or invalid access token)	UNAUTHORIZED	401
Email already verified — reason `already_verified`	CONFLICT	409
Asked again within 60 seconds — `error.details.retryAfterSeconds` says how long to wait	TOO_MANY_REQUESTS	429

8.2 PATCH /api/v1/auth/email-verification

Verifies an email address with the token from the emailed link. No session is needed, because the link may be opened on another device. Each link works once.

Request

Field	Value
Method	PATCH
Auth required	No — the token is the proof
Body fields	`token` (string, required)

Example request

```
PATCH /api/v1/auth/email-verification

{ "token": "<token from the emailed link>" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": { "emailVerified": true }
}
```

Response — link expired

```
400 Bad Request
{
  "success": false,
  "message": "This link has expired — please request a new one",
  "error": {
    "code": "BAD_REQUEST",
    "details": { "reason": "token_expired" }
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Token unknown, or issued for a different purpose — reason `token_invalid`	BAD_REQUEST	400
Link past its lifetime — reason `token_expired`	BAD_REQUEST	400
Link already used — reason `token_used`	BAD_REQUEST	400

8.3 POST /api/v1/auth/password-reset

Forgot password. Emails a password reset link. The response is always 200 with the same message, whether or not the email has an account, so it can't be used to find out who is registered. A link is only sent to an existing account that isn't Banned or Declined, at most once a minute (a quicker repeat sends nothing). The link expires in 1 hour and replaces any earlier unused one.

Request

Field	Value
Method	POST
Auth required	No — public endpoint
Body fields	`email` (string, required)

Example request

```
POST /api/v1/auth/password-reset

{ "email": "jane@example.com" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
`email` missing	VALIDATION_ERROR	422

8.4 PATCH /api/v1/auth/password-reset

Sets a new password with the token from the reset link. It signs out every session (the stored refresh token and the cookie are cleared), marks the email verified, because the link proves ownership, and emails a "password changed" notice.

Request

Field	Value
Method	PATCH
Auth required	No — the token is the proof
Body fields	`token` (string, required); `password` (string, required; the five password rules)

Example request

```
PATCH /api/v1/auth/password-reset

{ "token": "<token from the emailed link>", "password": "Secret123!" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": { "role": "Adopter" }
}
```

`role` lets the reset page link to the right login.

Error Cases

Scenario	Error Code	HTTP Status
Token unknown, or issued for a different purpose — reason `token_invalid`	BAD_REQUEST	400
Link past its lifetime — reason `token_expired`	BAD_REQUEST	400
Link already used — reason `token_used`	BAD_REQUEST	400
Password too weak — `error.details.unmet` lists the missing rules	VALIDATION_ERROR	422

8.5 PATCH /api/v1/auth/password

Changes the password of a signed-in user. It needs the current password. This session stays signed in (a new refresh token cookie is set) and any other session is signed out. A "password changed" notice is emailed.

Request

Field	Value
Method	PATCH
Auth required	Yes — Bearer access token, through `authenticate.allowPending`, so a Pending Staff member, Veterinarian or Volunteer can change it while they onboard
Body fields	`currentPassword` (string, required); `newPassword` (string, required; same rules as a reset, and must differ from the current one)

Example request

```
PATCH /api/v1/auth/password
Authorization: Bearer <accessToken>

{ "currentPassword": "Secret123!", "newPassword": "Fresh456?" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401
Current password is wrong — reason `wrong_password`	UNAUTHORIZED	401
New password too weak — `error.details.unmet`	VALIDATION_ERROR	422
New password is the same as the current one	VALIDATION_ERROR	422

8.6 POST /api/v1/auth/reactivation

Emails a link to reopen an account its owner closed. The response is always 200 with the same message. A link is only sent when the email belongs to a Closed account (closed by its owner) — never to one an admin or staff member deactivated, one declined at sign-up or a banned one — at most once a minute. The link expires in 24 hours.

Request

Field	Value
Method	POST
Auth required	No — public endpoint
Body fields	`email` (string, required)

Example request

```
POST /api/v1/auth/reactivation

{ "email": "jane@example.com" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
`email` missing	VALIDATION_ERROR	422

8.7 PATCH /api/v1/auth/reactivation

Reopens a Closed account with the token from the emailed link. Adopter and Donor accounts go straight back to Active; Staff, Veterinarian, Volunteer and Admin accounts go back to Pending and are re-approved the usual way. It also marks the email verified.

Request

Field	Value
Method	PATCH
Auth required	No — the token is the proof
Body fields	`token` (string, required)

Example request

```
PATCH /api/v1/auth/reactivation

{ "token": "<token from the emailed link>" }
```

Response — Adopter

```
200 OK
{
  "success": true,
  "message": "...",
  "data": { "role": "Adopter", "accountStatus": "Active" }
}
```

Response — Staff

```
200 OK
{
  "success": true,
  "message": "...",
  "data": { "role": "Staff", "accountStatus": "Pending" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Token unknown, or issued for a different purpose — reason `token_invalid`	BAD_REQUEST	400
Link past its lifetime — reason `token_expired`	BAD_REQUEST	400
Link already used — reason `token_used`	BAD_REQUEST	400
The account is no longer Closed — reason `not_closed`; the link stays unused	CONFLICT	409

8.8 POST /api/v1/auth/email-change

Asks to change the login email. It needs the current password. A confirmation link goes to the new address and a heads-up to the current one; nothing changes until the link is opened. The link expires in 24 hours and a newer request replaces it.

Request

Field	Value
Method	POST
Auth required	Yes — Bearer access token, through plain `authenticate` (a Pending account gets 401)
Body fields	`newEmail` (string, email format, at most 45 characters, required); `currentPassword` (string, required)

Example request

```
POST /api/v1/auth/email-change
Authorization: Bearer <accessToken>

{ "newEmail": "jane.smith@example.com", "currentPassword": "Secret123!" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": null
}
```

Response — asked again too soon

```
429 Too Many Requests
{
  "success": false,
  "message": "An email was just sent — please wait
              42 seconds before asking for another",
  "error": {
    "code": "TOO_MANY_REQUESTS",
    "details": { "retryAfterSeconds": 42 }
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401
Current password is wrong — reason `wrong_password`	UNAUTHORIZED	401
Another account already uses that address — reason `email_taken`	CONFLICT	409
`newEmail` missing, invalid, longer than 45 characters, or the same as the current one	VALIDATION_ERROR	422
Asked again within 60 seconds — `error.details.retryAfterSeconds`	TOO_MANY_REQUESTS	429

8.9 PATCH /api/v1/auth/email-change

Confirms an email change with the token from the link. No session is needed. It re-checks that the address is still free, sets it as the login email (verified) and tells the old address that it changed.

Request

Field	Value
Method	PATCH
Auth required	No — the token is the proof
Body fields	`token` (string, required)

Example request

```
PATCH /api/v1/auth/email-change

{ "token": "<token from the emailed link>" }
```

Response

```
200 OK
{
  "success": true,
  "message": "...",
  "data": { "userEmail": "jane.smith@example.com" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Token unknown, or issued for a different purpose — reason `token_invalid`	BAD_REQUEST	400
Link past its lifetime — reason `token_expired`	BAD_REQUEST	400
Link already used — reason `token_used`	BAD_REQUEST	400
Someone else took the address in the meantime — reason `email_taken`	CONFLICT	409

8.10 The email_unverified gate

Two existing endpoints now require a verified email: `POST /adoption-applications` and `POST /donations/checkout`. Everything else works unverified. The check runs before any pet or shelter lookup and before any Stripe call.

Response — email not verified

```
409 Conflict
{
  "success": false,
  "message": "...",
  "error": {
    "code": "CONFLICT",
    "details": { "reason": "email_unverified" }
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
`POST /adoption-applications` with an unverified email — reason `email_unverified`	CONFLICT	409
`POST /donations/checkout` with an unverified email — reason `email_unverified`	CONFLICT	409

> ***409, not 403:*** *the frontend's axios interceptor sends every 403 to `/forbidden`. Login refusals are likewise 401, never 403.*

8.11 Changes to existing endpoints

Endpoint	Sprint 9 change
`POST /auth/register`	Sends the first verification link. The password must meet the five rules (422 with `error.details.unmet` otherwise).

Endpoint	Sprint 9 change
`POST /auth/login`	`user.emailVerified` is returned for every role. A Closed account is refused with 401 and `error.details.reason = "account_closed"`; a Declined one with `"account_declined"`. Both are refused after a correct password.
`POST /auth/refresh-token`	`user` has the same `emailVerified` field as login.
`DELETE /<role>/me`	Mode `deactivate` now sets the account to Closed, which its owner can undo by emailed link.
`authenticate.allowPending` routes	Now include `POST /auth/email-verification` and `PATCH /auth/password`, so a Pending Staff member, Veterinarian or Volunteer can verify and change their password while onboarding.

9. Configuration and Setup

9.1 Environment variables

Set these in `server/.env`. Email switches on only when **both** `EMAIL_API_KEY` and `EMAIL_FROM` are set; with either blank, every email is printed to the server console instead. `.env` is loaded once at startup, so restart the server after changing it.

Variable	Needed for	Purpose
`EMAIL_API_KEY`	Real email	The Brevo API key. It starts with `xkeysib-`; the SMTP key (`xsmtpsib-...`) won't work.
`EMAIL_FROM`	Real email	The sender address. It must be verified in the Brevo account, or Brevo rejects the send.
`EMAIL_FROM_NAME`	Optional	The sender name. Defaults to `PetPals`.
`CLIENT_URL`	Links	The base of every link in an email. Defaults to `http://localhost:3000`.

9.2 Other email settings (config/email.js)

Constant	Value	Purpose
`TOKEN_TTL_MS`	24 h; reset 1 h	How long each emailed link works (Section 3.3)
`RESEND_COOLDOWN_MS`	60 seconds	Minimum gap between two links of the same purpose for a user
`SEND_TIMEOUT_MS`	10000	How long to wait for the provider before giving up
`DEFAULT_FROM_NAME`	`PetPals`	Sender name when `EMAIL_FROM_NAME` isn't set

9.3 Setting up Brevo

The full step-by-step is in `setup/SETUP.md`, under **Email (optional)**. In short:

1. **Sign up** at brevo.com and complete the account profile it asks for.
2. **Verify a sender:** Senders, Domains & Dedicated IPs → Senders → Add a sender, with `PetPals` as the name and an address you can receive mail at, then confirm the code Brevo emails to it. Recipients see this address.
3. **Create an API key:** SMTP & API → API Keys. Copy it straight away, because Brevo shows it only once.
4. **Authorized IPs:** newer accounts block API calls from addresses Brevo doesn't recognise. Under Security → Authorised IPs, add your IP or turn the blocking off. Turn it off for Render, whose outgoing IPs change.

5. **Configure and restart:** set `EMAIL_API_KEY`, `EMAIL_FROM` and `EMAIL_FROM_NAME`, restart the server, then register an account with your own address and check that the verification email arrives.

9.4 Limits and caveats

- **Free plan** — Brevo's free plan allows 300 emails a day.
- **Spam folder** — mail sent "from" a Gmail or Outlook address through Brevo often lands in spam, because those providers don't recognise Brevo as an allowed sender for their addresses. For a deployment, verify your own domain under Brevo's Domains tab and send from an address on it.
- **Dedicated sender** — recipients see the sender address, so use a dedicated one rather than a personal address once other people use the app.

9.5 Seeded logins and real email

The seed covers every account status for every role, including `*.closed@` and `*.declined@` accounts, an unverified adopter and donor (`adopter.unverified@`, `donor.unverified@`) and one spent token per purpose. The seeded addresses are fake `@petpals.com` ones that don't exist. With a real email key set, approving, declining or resetting them sends mail that bounces, and bounces count against your Brevo sender reputation. Never act on the seeded logins with real email on; leave the key blank and use the console fallback instead.

9.6 If an email doesn't arrive

The server terminal logs a line starting with `[email] failed to send`, with Brevo's reason, and Brevo's Transactional → Logs page shows what it received. A failed email never fails the request that triggered it.

10. Testing

10.1 Unit tests

Unit suites mock `services/email`, so a suite that asserts on email checks what would have been sent instead of sending it. Prisma is fully mocked in the unit tests, as everywhere else.

10.2 No test reaches the provider

`tests/setup/noEmailProvider.js` runs before every test file (Jest `setupFiles`) and blanks the `EMAIL_*` variables, so a real key in `server/.env` is never used by a test. `sendEmail` also prints nothing when `NODE_ENV` is `test`. `noAiProvider.js` does the same for the `AI_*` variables.

10.3 Integration journeys

`tests/integration/auth/sprint9Journeys.test.js` captures the emails through a mocked `services/email` and follows their links, against the live database. Like the other integration suites, it seeds its data through the API, cleans up through Prisma in `afterAll` and runs with `--runInBand`.

Because a journey reads each link out of a captured email and then calls the endpoint behind it, it exercises the whole chain a person goes through for a flow in Section 4: the email, the link in it and the request that redeems it. The email-verification gate on applying also gets a Cypress spec in Sprint 10.